

StegaVault

Follow-Up Audit: Scope Gaps vs. Phase-1 Objectives

Prepared for: Naveen K | Compares current source code against your Phase-1 SecureCloud objectives & scope

This document compares your Phase-1 objectives and scope (encryption, session auth, secure sharing, access control, audit logging) against what is *actually implemented* in the current StegaVault source code today. It found one issue in the same category as the original fake-encryption problem — a feature that looks fully functional in the UI but does nothing real underneath — plus several features from your predecessor system (SecureCloud) that did not carry over. Hand this document to your coding agent (e.g. Antigravity) one issue at a time, in priority order.

#	Issue	Severity	Est. effort
1	Share Manager is fully UI-mocked — fake “presigned” links, no backend	CRITICAL	15 min (relabel)
2	Graphical password auth — present in predecessor, absent here	MEDIUM	Descope: 5 min
3	Multi-factor auth (MFA) — schema ready, zero implementation	MEDIUM	Descope: 5 min
4	Role-based access control — schema ready, unused everywhere	LOW	Descope: 5 min
5	Audit log persists locally only, not to the backend schema	LOW	5 min (docs)

Bottom line

- Issue 1 is the only one that needs action before you present — it’s the same class of risk as the original fake encryption: something a panelist could click on and discover is hollow. It takes 15 minutes to make honest.
- Issues 2–5 are not bugs — they’re features your Phase-1 scope mentioned that this build doesn’t cover. The fast, safe move given your timeline is to explicitly descope them in your README/slides rather than attempt to build them now.

ISSUE 1: Share Manager Is Fully UI-Mocked

CRITICAL

The problem

The app has a complete “Share Manager” tab — a form to create a “Presigned Share Link” with a recipient email and expiry, plus a list of “Active Presigned Share Links” showing access counts. None of it is real. In `apps/web/src/App.tsx`, clicking “Create” writes a plain JavaScript object to `localStorage` with a hardcoded fallback recipient (`external@partner.io`) and a made-up access count. The log entry it writes literally says the link was “**dispatched to**” the recipient — but no email, notification, or network request of any kind is sent. No backend endpoint for sharing exists (confirmed: only OAuth routes exist on the backend, same as the original audit found for file storage). The term “presigned” specifically implies an AWS S3 presigned URL, which nothing in the app generates.

Why it must be fixed

- This is functionally the same risk as the original fake-encryption issue: a feature that looks complete and convincing in the UI but does nothing real. If a panelist enters their own email to “test” the share feature, expecting to receive something, nothing will happen — and the log claiming it was “dispatched” makes that worse, not better.
- Your Phase-1 objectives explicitly listed “encrypted file sharing between team members” and “role-based access control per file/folder” as core scope — so unlike the MFA/graphical-password items below, this one can't be waved off as predecessor-only scope; it's scope you already committed to for this project.

Step-by-step fix — Path A (recommended, ~15 minutes, do this regardless of time left)

Given file storage itself isn't wired to a real backend yet (see the earlier architecture audit), a real share feature can't exist independently of that — you can't share a file that isn't really stored anywhere. The correct move before presenting is to make the UI honest about being a prototype, exactly like the architecture slide already does for backend storage.

Step 1. In `apps/web/src/App.tsx`, find the three strings below and reword them to stop implying real delivery:

```
// Current (misleading):
addLog('SHARE_INIT', `Initiated presigned share link creation for "${file.name}"`);
...
addLog('SHARE_CREATE', `Presigned share link for "${activeShareFile?.name}" ` +
`dispatched to ${recipient} - expires in ${shareExpiry}`);

// Reworded (honest - no claim of real delivery):
addLog('SHARE_INIT', `Prototype share link created locally for "${file.name}" ` +
`(not yet sent - backend delivery not implemented)`);
...
addLog('SHARE_CREATE', `Prototype share link for "${activeShareFile?.name}" ` +
`saved locally for ${recipient} - no email or network request is sent yet`);
```

Step 2. Add a small, visible badge or note in the Share Manager tab UI itself — e.g. “Prototype: links are stored locally in your browser only and are not sent to recipients yet” — so it's honest even if a panelist doesn't read logs.

Step 3. Remove or rename the word “Presigned” everywhere in the Share Manager UI (button label, headings, modal title) — that term specifically claims an S3-backed mechanism that doesn't exist. “Share Link (Prototype)” or “Local Share Record” is accurate.

Step-by-step fix — Path B (only if you have real spare time, this depends on Issue 3 from the earlier audit)

Step 1. This requires the file itself to be persisted server-side first (S3 + the existing **File** Prisma model) — sharing a file that only exists in the browser's memory isn't meaningful.

Step 2. Once a file is really stored, add a minimal **POST /files/:id/share** endpoint using the **Share** model that's already fully designed in your Prisma schema (fields for **recipientEmail**, **tokenHash**, **permission**, **expiresAt** all already exist — nothing to design, only to wire up).

Step 3. Send a real email (e.g. via a transactional email API) containing a link with the generated token, and add a `GET /shares/:token` endpoint to redeem it.

Step 4. Given your timeline, treat this as future work — Path A is what should ship before the review.

ISSUE 2: Graphical Password Authentication — Dropped

MEDIUM

The problem

Your own Existing System slide describes SecureCloud (the predecessor project) as having “graphical password authentication with role-based access control.” StegaVault does not carry this forward: the backend schema still has a `graphicalPasswordHash` field on the `User` model, but nothing in the frontend or backend implements it — confirmed by a full source search finding zero references anywhere in `apps/web/src`.

Why it matters

- It's not a bug — StegaVault made a deliberate, reasonable choice to use OAuth instead. But if a panelist compares your Existing System slide to your Proposed System and asks “did you keep the graphical password login,” you want a ready, confident answer rather than being caught off guard.

Recommended fix — descope explicitly (5 minutes)

Step 1. Add one line to your README's “Current scope vs. roadmap” section (the one already added for backend storage): “Graphical password authentication, used in the SecureCloud predecessor, was superseded by OAuth (Google/Discord/LinkedIn) in this version for stronger, more standard security. The schema field is retained for potential future multi-factor use.”

Step 2. In the viva, if asked, give exactly that answer — it reads as a deliberate design decision, not an oversight.

ISSUE 3: Multi-Factor Authentication (MFA) — Not Implemented

MEDIUM

The problem

The `User` model has `mfaTotpSecret` and `mfaEnabled` fields, but no TOTP enrollment, QR code generation, or verification logic exists anywhere in the codebase.

Why it matters

- Same reasoning as graphical password: not a defect, but worth a ready answer if asked, especially since your Future Scope slide already mentions security hardening as a direction.

Recommended fix — descope explicitly (5 minutes)

Step 1. Add MFA to your existing Future Scope slide/README list: “TOTP-based multi-factor authentication (schema already supports it via `mfaTotpSecret`/`mfaEnabled`) — planned once backend authentication is wired up.”

Step 2. If you do want to actually build it later (not before this review): use a library like `speakeasy` or `otplib` server-side to generate/verify TOTP codes, and a QR code library client-side for enrollment.

ISSUE 4: Role-Based Access Control — Unused

LOW

The problem

The **User** model has a **Role** enum (**user** / **admin**), but nothing in the app reads or enforces it — there's no admin view, no permission check gating any action.

Why it matters

- Lowest priority of this batch: your current single-user personal-vault use case doesn't structurally need admin roles yet. Still worth a one-line mention so it doesn't read as a forgotten requirement.

Recommended fix — descope explicitly (5 minutes)

Step 1. Add to Future Scope: “Role-based access control (admin/user roles already modeled in the schema) — planned for a future multi-user/team deployment.”

ISSUE 5: Audit Log Is Real, But Local-Only

LOW

The problem

Unlike the Share Manager, this one is mostly honest: the Audit Logs tab genuinely records real user actions (encrypt, share, etc.) as they happen — it isn't faked data. But it only writes to **localStorage**, so logs vanish if the browser data is cleared or a different device is used, even though the backend schema already has a dedicated **AccessLog** table designed for exactly this.

Why it matters

- Not misleading like Issue 1 — what's shown did happen. It's simply not durable yet, consistent with the rest of the app's “crypto/stego real, backend persistence is next milestone” story you're already telling.

Recommended fix (5 minutes)

Step 1. Add one line next to your existing backend-storage roadmap note: “Audit logs are currently recorded client-side only; persisting them to the backend AccessLog table is part of the same storage-integration milestone.”

Step 2. No code change needed before the review — this is purely a documentation/talking-point fix so the scope is fully accurate everywhere a panelist might look.

Priority order: fix Issue 1 first (15 minutes, prevents a live-demo surprise). Then add the four short descope notes for Issues 2–5 to your README/slides (20 minutes total) so every feature your project touches on is described accurately, whether implemented, prototyped, or planned.